PuppyRaffle Protocol Audit Report

Date: July 12, 2026

Prepared by: Audit Team

Lead Auditors: Vraj Parikh

Table of Contents

- PuppyRaffle Protocol Audit Report
- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
    1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Audit team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

** The findings described in the section are from the following commit hash:

```
2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
./src/
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

Notes about how the audit went, how many hours spent, etc.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	1
Info	6
Gas Optimizations	2
Total	15

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain the raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) pattern and as a result, allow participants to drain the contract balance

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not

@>      payable(msg.sender).sendValue(entranceFee);
@>      players[playerIndex] = address(0);
```

```
        emit RaffleRefunded(playerAddress);
    }
```

A playwer who has entered raffle could have a fallback/receive function that cause the `PuppyRaffle::refund` function again and claim another refund. They could continue this cycle till the contract balance is drained

Impact: All fees paid by raffle entrants could be stolen by the malicious participant

Proof of Concept:

1. User enters raffle
2. Attacker sets up the contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance

Prrof of Code:

▶ Code

Recommended Mitigation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emmission up as well.

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not

+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);

        payable(msg.sender).sendValue(entranceFee);

-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
    }
```

[H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows user to influence or predict the winner and influence the winning puppy

Description: Hashing msg.sender, block.timestamp and block.difficulty together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of raffle themselves.

Note: This means users can front-run this function and call `refund` function if they see they are not the winner

Impact: Any user can influence the winner of the raffle winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See this solidity blog on prevrandao. `block.difficulty` was recently replaced with `prevrandao`
2. User can mine/manipulate their `msg.sender` value to results in the address being used to generate a winner
3. User can revert their `selectWinner` transaction if they don't like the winner or resulting puppy

Using on-chain values as a random number is a [well-documented attack vector](https://medium.com/better-programming/how-to-generate-truly-random-numbers-in-solidity-and-blockchain-9ced6472dbdf) in the blockchian space

Recommended Mitigation: Consider using a cryptographically provable random number genrator such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to `0.8.0`, integers were subject to integer overflows

```
uint64 myVar = type(uint64).max;
// myVar will be 18446744073709551615
myVar = myVar + 1;
// myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are a accumulated for the `feeAddress` to collect later in the `PuppyRaffle::withdrawFees`. However, if the `totalfees` variable overflows, the `feeAddress` might not collect the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept:

1. We conclude a raffle of 4 players
2. We then have 89 players to enter a new raffle and conclude the raffle
3. `totalFees` will be :

```
totalFees = totalFees + uint64(fee)
// substituted
totalFees = 800000000000000000 + 17800000000000000000;
```

```
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

4. You will not be able to withdraw due to the line in `PuppyRaffle::withdrawFees`

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently player
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdrw the fees, this is clearly not the intended design of the protocol. At some point, there will be too much baalnce in the contract that the above will be impossible to hit.

▶ Code

Recommended Mitigation: There are a few possible mitigations:

1. Use a newer version of solidity, and a `uint256` instead of `uint64` for `PuppyRaffle::totalfees`
2. You could also use `SafeMath` library of openzeppelin for version 0.7.6 of solidity. However you would still have a hard time with the `uint64` if too many fees are collected.
3. Remove the balance check from `PuppyRaffle::withdrawfees`

```
- require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently play
```

There are more attack vectors with that final require, so we recommend removing it regardless

Medium

[M-1] Looping through the array to check out duplicates in `PuppyRaffle::enterRaffle` is a potential Denial of Service (DoS) attack, incrementing the gas costs for future entrants

Impact: Medium Likelyhood: Medium

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer `PuppyRaffle::players` array is, the more checks a new player has to make. That means the gas costs for players who enter right when raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array is an additional check the loop will have to make

```
// @audit DoS attack
        for (uint256 i = 0; i < players.length - 1; i++) {
            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
```

```
            }
        }
```

Impact: The gas costs for Raffle entrants will greatly increase as more players enter the raffle, discouraging later users from entering, and causing a rush at the start ofn a raffle to be one of the first entrants in the queue

An attacker might make the `PuppyRaffle::entrants` array so big that noone else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such

- First 100 players: 6252039
- Second 100 players: 18067741

This is more than 3x expensive for second 100 players

▶ POC

Recommended Mitigation: There are few recommendations:

1. Consider allowing duplicates. Users can make new wwallet addresses so a duplicate check doesn't prevent the same person from entering raffle multiple times, only the same wallet address
2. Consider using mappings to check for duplicates. This would allow constant time lookup of whether a user has already been entered

```
+    mapping(address => uint256) public addressToRaffleId;
+    uint256 public raffleId = 0;
    .
    .
    .
    function enterRaffle(address[] memory newPlayers) public payable {
        require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough
        for (uint256 i = 0; i < newPlayers.length; i++) {
            players.push(newPlayers[i]);
+            addressToRaffleId[newPlayers[i]] = raffleId;
        }

-        // Check for duplicates
+       // Check for duplicates only from the new players
+       for (uint256 i = 0; i < newPlayers.length; i++) {
+          require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle: Duplicate play
+       }
-        for (uint256 i = 0; i < players.length; i++) {
-            for (uint256 j = i + 1; j < players.length; j++) {
-                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
```

```
-            }
-        }
        emit RaffleEnter(newPlayers);
    }
.
.
.
    function selectWinner() external {
+       raffleId = raffleId + 1;
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not
```

Alternatively, you could also use [Openzeppelin's `Enumerable` set library]

[M-2] Unsafe cast of PuppyRaffle::fee loses fees

Description: In `PuppyRaffle::selectWinner` their is a type cast of a uint256 to a uint64. This is an unsafe cast, and if the uint256 is larger than type(uint64).max, the value will be truncated.

```
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not
        require(players.length > 0, "PuppyRaffle: No players in raffle");

        uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
        address winner = players[winnerIndex];
        uint256 fee = totalFees / 10;
        uint256 winnings = address(this).balance - fee;
@>      totalFees = totalFees + uint64(fee);
        players = new address[](0);
        emit RaffleWinner(winner, winnings);
    }
```

The max value of a uint64 is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the fee casting will truncate the value.

Impact: This means the feeAddress will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the fee as a uint64 hits
3. totalFees is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0
```

Recommended Mitigation: Set `PuppyRaffle::totalFees` to a uint256 instead of a uint64, and remove the casting. Their is a comment which says:

// We do some storage packing to save gas But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
-   uint64 public totalFees = 0;
+   uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not
        require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty)
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-       totalFees = totalFees + uint64(fee);
+       totalFees = totalFees + fee;
```

[M-3] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.

2. The lottery ends
3. The selectWinner function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owness on the winner to claim their prize. (Recommended)

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` function returns 0 for noon-existent players and for player at index 0, causing a palyer at index 0 to think incorrectly that they have not entered the raffle

Description: If a player is in `PuppyRaffle::players` array at index 0, this will return 0 but according to natspec, it will also return 0 if player is not in the array.

```
    /// @return the index of the player in the array, if they are not active, it returns 0
    function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
    }
```

Impact: A player at index 0 may incorrectly think they have entered the raffle and attempt to enter the raffle again, wasting gas

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to function documentation

Recommended Mitigation: Easiest recommendation would be to revert if the player is not in the array instead of returning 0

You could also reserve the 0th index for any competition, but a better solution might be to return an `int256` where the function might return -1, if the player is not active

Gas

[G-1] Unchanged state variables should be declared constant or immutable

Reading from storage is much more exoensive than reading from a constant or immutable variable

Instances:

- `PuppyRaffle::raffleDuration` should be immutable
- `PuppyRaffle::commonImageUri` should be constant
- `PuppyRaffle::rareImageUri` should be constant
- `PuppyRaffle::legendaryImageUri` should be constant

[G-2] Storage variables in loop should be cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient

```
+       uint256 playersLength = players.length;
-       for (uint256 i = 0; i < players.length - 1; i++) {
+       for (uint256 i = 0; i < playersLength - 1; i++) {
-           for (uint256 j = i + 1; j < players.length; j++) {
+           for (uint256 j = i + 1; j < playersLength; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
            }
        }
```

Informational

[I-1]: Solidity Pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in src/PuppyRaffle.sol Line: 2

```
pragma solidity ^0.7.6;
```

[I-2] Using an outdated version of solidity is not recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: Deploy with a recent version of Solidity (at least `0.8.0`) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see [slither] (https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity) documentation for more information

[I-3]: Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

- Found in src/PuppyRaffle.sol (src/PuppyRaffle.sol#L65)

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol (src/PuppyRaffle.sol#L187)

```
        feeAddress = newFeeAddress;
```

[I-4] Use of “magic” numbers is discouraged

It can be confusing to see number literals in codebase and it is much more readbale if the numbers are given a name.

Examples:

```
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;

        Instead you could use:
        uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
        uint256 public constant FEE_PERCENTAGE = 20;
        uint256 public constant POOL_PRECISION = 100;
```

[I-5] State Change Without Event

There are state variable changes in this function but no event is emitted. Consider emitting an event to enable offchain indexers to track the changes.

- Found in src/PuppyRaffle.sol Line: 157

```
    function withdrawFees() external {
```

[I-6] Dead Code

Functions that are not used. Consider removing them.

- Found in src/PuppyRaffle.sol Line: 173

```
    function _isActivePlayer() internal view returns (bool) {
```